

# CircuitHub Allegro Bridge SDK 1.9.0

---

This is the private distribution companion for `CircuitHub.AllegroBridge.Sdk` 1.9.0. The release workflow publishes the exact prebuilt NuGet package staged in the private GitHub draft release; it does not rebuild or substitute the native-accepted package.

## Download

You need access to the private `elkowalski1989/allegro-bridge` repository and an authenticated GitHub CLI:

```
gh auth login
gh release download v1.9.0 `
  --repo elkowalski1989/allegro-bridge `
  --pattern "CircuitHub.AllegroBridge.Sdk.1.9.0.nupkg" `
  --dir C:\CircuitHub\AllegroBridge\1.9.0
gh release download v1.9.0 `
  --repo elkowalski1989/allegro-bridge `
  --pattern "circuithub_allegro_bridge-1.9.0-py3-none-win_amd64.whl" `
  --dir C:\CircuitHub\AllegroBridge\1.9.0
```

The same release carries this guide as Markdown, standalone HTML, and PDF. It also carries the internal publisher certificate, its trust explanation, and scoped `CurrentUser` installation/removal scripts.

Release: <https://github.com/elkowalski1989/allegro-bridge/releases/tag/v1.9.0>

## Verify the package

The package must have this SHA-256 before use:

```
CircuitHub.AllegroBridge.Sdk.1.9.0.nupkg
SHA-256: d81acfad5550b668f2c2c062735c5813537dff5038da8fc8e5b058233b55f8cf

circuithub_allegro_bridge-1.9.0-py3-none-win_amd64.whl
SHA-256: 8bcab4d67dee4def1ba69329110b145260120a062b0f87ec04cab84cd3596e35
```

In PowerShell:

```
(Get-FileHash `
  -Path "C:\CircuitHub\AllegroBridge\1.9.0\CircuitHub.AllegroBridge.Sdk.1.9.0.nupkg" `
  -Algorithm SHA256).Hash.ToLowerInvariant()
```

The signed NativeAOT host embedded in the package has SHA-256 `9b7c4f8aabc17855769bc357eb710575e1b8fc6962cf903630eb55a45c982b68`. The package's own NuGet metadata records source commit [2e0400efb2a204361af31be3fadb9921491040db](#).

## Use a project-local package source

Keep the downloaded package in a controlled folder and add a project-local `NuGet.Config` beside the consuming solution:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <clear />
    <add key="CircuitHubPrivate" value="C:\CircuitHub\AllegroBridge\1.9.0" />
  </packageSources>
</configuration>
```

Reference the exact version from the consuming project:

```
<ItemGroup>
  <PackageReference Include="CircuitHub.AllegroBridge.Sdk" Version="1.9.0" />
</ItemGroup>
```

Then restore and build normally:

```
dotnet restore
dotnet build -c Release
```

The package's transitive build target copies the signed host and package-owned resident SKILL files to the application output. No Allegro Bridge source checkout or `ProjectReference` is required.

## Use from Python

Install the exact private Windows x64 wheel directly from the downloaded file:

```
py -3.12 -m pip install `
"C:\CircuitHub\AllegroBridge\1.9.0\circuithub_allegro_bridge-1.9.0-py3-none-
win_amd64.whl"
```

The wheel contains the same signed host, managed SDK assemblies, and resident SKILL bytes as `CircuitHub.AllegroBridge.Sdk.1.9.0.nupkg` ; publication validates those bytes before upload. Python uses the public managed SDK through Python.NET and does not implement the private broker framing, licensing, authorization, or stale-state rules.

```

import asyncio

from circuithub_allegro_bridge import BridgeSession

async def main() -> None:
    async with await BridgeSession.connect_and_wait_until_ready(
        r"C:\Users\engineer\AppData\Local\Temp\pdbridgea12345"
    ) as session:
        receipt = await session.execute_to_terminal("board.refresh")
        print(receipt.state.value, receipt.message)

asyncio.run(main())

```

The source repository includes a tested installable consumer at `samples/PythonSdkQuickstart`. Its `allegro-bridge-quickstart` command accepts one explicit active `pdbridge...` session directory, runs a safe `board.refresh`, prints one JSON result line on success, and exits nonzero on a version, readiness, entitlement, design, operation, or session-fencing failure. C# clients use `ExecuteToTerminalAsync` or `ExecuteWithHandleAsync`; Python clients use `execute_to_terminal` or `execute_with_handle` plus `wait_for_terminal`. A wait timeout does not cancel or replay the underlying operation; cancellation is explicit on the operation handle.

Contract-3 custom-SKILL operations additionally expose a single bounded typed feedback stream and phase-registered actions. C# uses `IAlegroOperationHandle.ReadInteractionFeedbackAsync` and `SendInteractionActionAsync`; Python uses `OperationHandle.read_interaction_feedback` and `OperationHandle.send_interaction_action`. Pointer updates coalesce latest-wins, semantic transitions remain ordered, terminal state closes the stream, and no Allegro database ID crosses the boundary.

For a live custom-SKILL lifecycle check, use the source revision recorded by this package, not a newer working branch. The source repository includes `tests/AllegroBridge.LifecycleLiveGate`. Build it with `UsePublishedAllegroBridgeSdk=true` and this exact package version, then give it one explicit active bridge directory and one new receipt path. It proves a generic `AwaitingUserInput` phase and terminal callback without UI Automation, journal parsing, operator clicks, or board mutation. The packaged `docs/Unsigned-SKILL-Extensions.md` file contains the complete command.

## SDK 1.9 upgrade notes and acceptance scope

SDK 1.9 adds `ObserveCanvasViews()` / `observe_canvas_views()` for bounded native view renewal and a foreground-independent display path: `CaptureCanvasDrawingView(view, presentationWindowHandle)` / `capture_drawing_view(presentation_window_handle=hwnd)`.

Keep observation running when another application has focus. The renderer must use the complete returned clip-rectangle union, revalidate every displayed frame, and clear on expired evidence, changed board/session, minimize, full coverage or unavailability. The rendering HWND must belong to the consumer process, be owned by the bound Allegro frame, exactly match the physical canvas bounds, and be layered, input-transparent, nonactivating and non-topmost. The SDK never activates or reorders windows. Pointer/picking and the legacy whole-frame drawing overload retain their foreground requirement.

The [short C#/Python consumer example](#) shows the supported integration. PD Simple's controller still needs to adopt this path; the release does not claim that PD Simple itself was modified.

Native Allegro 25.1 P001 acceptance covered focus changes, overlapping windows, full-cover hiding, ongoing clipped rendering, zoom/pan/fit, move/resize, minimize/restore and 96 -> 144 -> 96 DPI transitions. The 10 completed native captures measured 75 targets and 66 negative controls within one horizontal and two vertical physical pixels (limit: two pixels per axis). The renderer consumed 15 successive native updates. Both monitors were restored to 100% after the test.

The native capture and pixel checks passed, but the launcher reported a separate process-exit timeout and terminated its disposable Allegro process after native target rollback. The process was confirmed gone and no board was saved. This is not represented as a clean launcher-exit pass. The protected host retains the existing entitlement and authorization implementation and is versioned and signed with the existing publisher identity for this release.

## Retained SDK 1.8 baseline

SDK 1.8 uses protocol 25 / custom-SKILL contract 3. The published 1.7.0 package and wheel do **not** supply the following additions. Use them only with a matching SDK, Windows assembly, signed host and package-owned resident from an accepted candidate, then start a fresh resident session. Consult the packaged specification and acceptance scope below; consumer rendering and engineering behavior remain separate from SDK acceptance.

| Development workflow               | C#                                                                    | Python                                                    |
|------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------|
| Retain accepted observation        | <code>CaptureObservation()</code>                                     | <code>capture_observation()</code>                        |
| Bind exact extension/commands      | <code>SkillExtensions.LoadAndBindAsync(...)</code>                    | <code>load_and_bind_skill_extension(...)</code>           |
| Prepare one execution attempt      | <code>PrepareProposalAsync(...)</code> → <code>StartAsync(...)</code> | <code>prepare_proposal(...)</code> → <code>start()</code> |
| Capture paired native/Windows view | <code>CaptureCanvasViewAsync(...)</code>                              | <code>capture_canvas_view(...)</code>                     |
| Require causal native feedback age | <code>TryGetInteractionFeedbackAge(...)</code>                        | <code>try_get_feedback_age_seconds(...)</code>            |
| Observe companion lifecycle        | <code>ActivationRequested</code> / <code>ResidentSessionEnding</code> | <code>subscribe_lifecycle()</code>                        |

Multiple clients use their ordinary readiness API against the same explicit active bridge directory. One current-user local broker owns the resident, lease, catalog and command admission; Python uses that same managed SDK. Only the submitting connection can cancel or send actions for its operation. Disposing an observer does not stop remaining clients. A disconnect does not prove rollback or transfer operation ownership to a new connection.

Readiness is an observation, not a reservation: the host still checks the session, board generation and snapshot identity before staging an extension. An unchanged board generation does not guarantee an unchanged snapshot hash. The resident's repeated-launch path only requests companion focus; it no longer republishes a snapshot for activation alone. Real selection, lifecycle, refresh and catalog changes still advance observations. The current host also reports the mismatched expected/observed identities on a stale preflight

rejection. Do not parse diagnostic text as an API or use blind retries to conceal rejection. The broker withholds retained snapshots until the resident acknowledges current liveness. A new readiness request after the native session stopped was rejected in installed-wheel Python acceptance. This correction does not imply that every possible startup heartbeat failure has the same cause.

Load/bind the extension before capturing the observation. Analyze outside Allegro while preserving scope, units, completeness and native provenance. Prepare/upload against that original observation; explicitly review before starting. An attempted proposal cannot be dispatched again, even after an uncertain outcome. A wait timeout is not cancellation or permission to retry. The native operation still validates engineering preconditions and returns readback/recovery evidence; `Complete` alone is not engineering acceptance.

Canvas identity and pointer classification are separate proofs. The paired view must still match session, board, viewport, windows, bounds, DPI and age when used. Native feedback requires a capture barrier retained **before** point/object acquisition. Receipt timestamps and replayed JSON do not prove acquisition age. Missing evidence means hidden positional graphics or neutral classification, never approximate alignment. PD owns rendering and routing policy; the SDK provides no native SKILL form or Options strip.

`TryGetCanvasPointer` separately supports neutral physical-pointer tracking through the latest inspected Windows canvas, even when native capture is unavailable or expired. It revalidates the live session, board generation, owner/surface, bounds, DPI and foreground; superseded inspections fail. It provides no native object verdict or board projection. Keep native labels, classification and positional graphics subject to their own freshness and alignment checks. A stable neutral cursor is not native alignment acceptance.

Live point projection also checks that the native child at the physical pixel matches the proven canvas. Disabled or input-transparent native controls may still hide pixels. Drawing projection rejects the whole frame if native controls or ancestor clipping obscure any canvas area; checking endpoints alone cannot protect a crossing line. Clear points elsewhere can remain usable. No monitor, board, title or Qt-class-specific coordinate adjustment is used.

SDK-only native acceptance on 2026-09-08 proved simultaneous C#/installed-wheel Python lifecycle, cancellation, observer-detach survival and stopped-resident readiness rejection. The later Windows-only correction passed ten native canvas views, including zoom/pan/fit, window changes and 96 → 144 → 96 DPI transitions. Independent hairlines measured 72 visible targets within one horizontal and two vertical physical pixels (accepted limit: two pixels per axis), with 66 clear empty controls. Three native-control-covered target cases were rejected; drawing was refused on obscured layouts and available on the unobscured secondary view. All ten owned-window captures were inspected and the temporary native geometry was read back exactly and rolled back without saving a board.

This evidence covers Allegro 25.1 P001 and the tested displays, not every native provider, selectable-object class or consumer UI. Native age was bounded to 250 ms; checks cannot promise instantaneous zoom-change awareness between calls. Honor viewport feedback, obtain fresh evidence and revalidate before use. A returned coordinate is not durable authority and unavailable graphics must hide. The Windows CI gate separately exercises owned-window occlusion and clipping; it does not replace actual Allegro pixel acceptance. The packaged development guide and specification contain the API examples and full scope.

A signed local development package is not by itself a publishable release. The publication workflow requires one consistent NuGet/Python version and the exact package source commit with successful CI. Preserve that boundary: do not rename a development archive to a released version or substitute older wheel assets. A native lifecycle pass proves only its recorded checks, not rendered canvas alignment or the consumer's engineering acceptance.

## Runtime and trust boundary

- C# consumers target .NET 10 on Windows. Python consumers require 64-bit Python 3.10 or newer, Python.NET 3.1.0, and the .NET 10 runtime.
- Cadence Allegro PCB Designer and a valid online CircuitHub Bridge entitlement are required for live operation.
- The modern WPF consumer is the sole rich GUI. Resident SKILL owns immediate Allegro board effects and does not provide a parallel native form.
- The host must retain a valid Authenticode signature. Use the release's trust instructions before installing the internal publisher certificate; do not weaken machine-wide trust or Windows security policy.
- Unsigned custom SKILL is arbitrary local code inside Allegro. Load only code you have reviewed and trust.

For the full typed API and integration contract, open `docs/AllegroBridge-SDK-Specification.html` from the package. For unsigned SKILL ownership and examples, read `docs/Unsigned-SKILL-Extensions.md` from the package.

## Support information

When reporting a package or runtime problem, include the SDK version, package SHA-256, source commit, host version, Allegro version, and the smallest relevant application-owned log excerpt. Do not include license keys, signing material, database IDs, or unrelated machine data.